

GENESIS 2.0

BUILDATHON GUIDELINES

1. Welcome to Genesis 2.0

"Great software isn't measured by how many technologies it uses, but by how well it is engineered."

Welcome to Genesis 2.0 Buildathon.

Genesis is not designed to be another hackathon where teams race to build the maximum number of features.

Instead, Genesis evaluates participants on their ability to design, develop, deploy, and justify software systems using modern engineering practices.

Throughout this Buildathon, teams are encouraged to think like software engineers rather than simply developers.

The competition rewards thoughtful engineering decisions, maintainable code, secure implementations, and scalable architectures.

Our objective is simple:

Don't just build software. Engineer it.

2. Competition Tracks

To ensure fair competition, participants compete only within their respective academic year.

There are three independent tracks:

- Second Year Track — Focuses on software engineering fundamentals.
- Third Year Track — Focuses on production-ready web applications.
- Fourth Year Track — Focuses on industry-standard engineering and cloud deployment.

Teams from different academic years will not compete against each other.

3. General Technical Requirements

These requirements apply to every participating team.

3.1 Source Code Management (Mandatory)

Git and GitHub are mandatory for every participating team.

The use of GitHub is considered a standard software engineering practice and is therefore required throughout Genesis Buildathon.

Requirements

- Create a dedicated GitHub repository.
- Maintain meaningful commit history.
- Organize the repository professionally.
- Include a comprehensive README.
- Use proper .gitignore files.
- Encourage contributions from all team members.

Judges may inspect:

- Commit history
- Repository quality
- Contributor activity
- Documentation
- Project organization

Failure to maintain a proper GitHub repository may result in score deductions or disqualification.

3.2 Official Development Window

To ensure fairness across all teams, implementation must begin only after the official commencement time.

Official Start

01 August 2026 — 10:00 PM IST

The following must not exist before this time:

- GitHub repository
- Local project directory
- Source code
- Backend implementation
- Frontend implementation
- Database implementation
- Configuration files
- Project-specific implementation

The organizing committee reserves the right to verify:

- Repository creation timestamps
- Commit history
- Local file timestamps
- Overall project timeline

Any evidence suggesting prior implementation may result in immediate disqualification.

4. Application Architecture Expectations

- Genesis encourages participants to build software using modern engineering practices.
- Teams are expected to develop dedicated frontend and backend applications.
- Applications should communicate using well-designed APIs such as REST or GraphQL.
- Participants may choose any programming language or framework unless otherwise specified for their track.

5. Important Note to Participants

The expectations outlined in this handbook may appear ambitious.

This is intentional. They represent the engineering practices commonly adopted in modern software development.

However, this handbook is not a mandatory checklist.

Teams are not expected to implement every technology or practice listed for their track.

Instead, we encourage participants to:

- Build a functional application first.
- Focus on correctness before complexity.
- Gradually adopt additional engineering practices.
- Learn throughout the Buildathon.

Projects are evaluated relative to other teams competing within the same academic year.

A project implementing fewer technologies exceptionally well will often outperform one that includes numerous technologies without proper understanding.

The objective is to build the best-engineered application within your track and not the most complicated one.

6. Additional Challenges

Genesis is more than an application development competition.

Throughout the event, participants may encounter surprise side quests.

These challenges will only be revealed during the event.

They may evaluate:

- Technical aptitude
- Collaboration
- Adaptability
- Communication
- Engineering skills
- Problem-solving

Participation may contribute towards:

- Overall evaluation
- Special recognitions
- Bonus awards

Stay prepared. Not every challenge will be announced beforehand.

7. Track Expectations

Second Year Track

Objective

Build a complete full-stack application demonstrating software engineering fundamentals.

Expected

- Functional full-stack application
- CRUD operations
- Authentication
- Password hashing
- Database integration
- Validation
- Error handling
- Responsive UI
- Git & GitHub
- Environment variables
- README

Bonus (Second Year Track)

- Deployment
- Search
- Pagination
- Image Upload
- Email Verification
- Role-Based Authentication

Third Year Track

Everything from Second Year, plus:

Deployment

- Vercel
- Render
- Railway

Managed Cloud Database

- MongoDB Atlas
- Supabase
- Neon
- Railway PostgreSQL

Cloud Storage

- Cloudinary
- Firebase
- Supabase Storage

Docker

- Docker
- Docker Compose

Authentication

- JWT
- Refresh Tokens
- OAuth (Google/GitHub)

Security

- Rate limiting

- Environment variables
- Password hashing
- Validation
- CORS

Architecture (any standard architecture)

- MVC
- Microservices
- Feature-Based Architecture

Fourth Year Track

Everything from Third Year, plus:

Cloud Infrastructure — AWS (or equivalent)

- EC2
- Elastic Beanstalk
- ECS
- App Runner

Database

- Amazon RDS
- Aurora

Storage

- Amazon S3

DevOps

- Docker
- Docker Compose
- CI/CD
- GitHub Actions

Security

- Secure Authentication
- Secure Cookies
- RBAC
- HTTPS
- Secrets Management

Performance

- Caching
- Pagination
- Query Optimisation
- Lazy Loading

Engineering — Teams should justify:

- Database selection
- Cloud architecture
- Deployment strategy
- Authentication
- Performance
- Scalability

8. Evaluation Philosophy

Genesis evaluates software engineering, not technology count.

Judges will assess:

- Correct implementation
- Code quality
- Architecture
- Security
- Deployment
- Engineering practices
- Technical understanding
- Maintainability

Using additional technologies does not automatically result in higher scores.

Projects will be evaluated based on:

- What was implemented
- How well it was implemented
- Whether the team understands their implementation

9. Final Note

Genesis 2.0 Buildathon has been designed to simulate modern software engineering rather than traditional feature-focused hackathons.

We encourage every participant to challenge themselves, collaborate effectively, experiment with new technologies, and understand every engineering decision made throughout the development process.

Whether your project implements five technologies or fifteen, what matters most is that your implementation is thoughtful, maintainable, secure, and technically sound.

Great software is not defined by the number of technologies it uses, but by the quality of its implementation.

We wish every participant the very best and look forward to seeing what you build.